What Actually Matters in Matrix-Structured Optimizers, and a Silent Failure in All of Them

Pop Alexandru · August 2026

Every number in this document was produced by the code in this repository, at the scale stated. Nothing is quoted from another paper as if it had been reproduced here. Where a claim failed, the failure is reported in the section that made it.

arXiv source: `docs/paper/paper.tex` · Code and exact commands: github.com/pop123-ux/knsa-knee-normality-kaggle (`astro/`).

ABSTRACT

Matrix-structured optimizers — Muon, Shampoo, SOAP, NorMuon — improve on AdamW by treating a weight matrix as a linear operator rather than a bag of coordinates. We report a controlled study of what in these methods is responsible for the gain, on nanoGPT's GPT-2, under a protocol with equal tuning budgets, ten seeds, exact paired rank tests, Holm correction, and wall-clock matching.

The central finding is structural rather than statistical. For a momentum matrix $M = U\Sigma V^\top$ with $m > n$, the row norms of Muon's update are exactly the *leverage scores* of M's row space. Two consequences follow that appear unremarked. First, for wide matrices the polar factor has orthonormal rows, so NorMuon's row normalization is provably inert there — measured participation 0.995 on GPT-2's 128×512 projection. Second, on a **fused** query/key/value projection the leverage localizes onto whichever stacked operator has the largest gradient: **85% of the squared row mass lands on V**, against 9% for K and 5% for Q, because Q and K reach the loss through the softmax Jacobian and receive gradients ~70× smaller. Two of the three operators inside the tensor receive almost no update, while the tensor passes every orthogonality check one would apply. Muon fails at the property it exists to provide, invisibly. Splitting the projection restores the mass to 0.336/0.334/0.330 and costs *less* than not splitting.

Combining that fix with cautious masking and NorMuon's placement beats **Muon, NorMuon and AdamW** on from-scratch GPT-2 at ten seeds, on a step budget *and* at matched wall-clock, with Holm-adjusted $p \le 0.03$ throughout. On a held-out fine-tuning task the split still helps directionally (8/10 seeds, $d = -0.62$) while the rest of the recipe does not transfer — reported as a limit rather than omitted.

We also report **four claims this work made and withdrew**, and **two bugs in our own measurement code**, both of which produced publishable-looking numbers. Every one was caught by a control disagreeing with a headline; none by reading the code. We argue this is the more transferable contribution: the standard fairness recipe of equal tuning budgets is necessary and demonstrably **not sufficient** — the hyperparameter *ranges* must match the update scale, which no published protocol we know of requires.

1. What this paper is

This is an account of an attempt to build an optimizer that beats the current state of the art, written so that the attempt can be checked rather than believed.

The motivation is specific. Wen et al. [1] benchmarked ten optimizers against a properly tuned AdamW and found the real speedups were near $1.1\times$ rather than the $2\times$ commonly claimed. The gap was not fraud; it was methodology — unequal tuning budgets and single-seed reporting. Any new optimizer paper therefore inherits an obligation: make the comparison hard to rig, and publish the protocol along with the numbers.

The second motivation is a gap in the literature. Matrix-structured optimizers are developed and validated for *from-scratch, large-batch pretraining*. Qu et al. [5] report that in the opposite regime — fully fine-tuning weights that were pretrained with Adam — Muon *loses* to Adam, because the optimizer mismatch disrupts pretrained knowledge in proportion to update strength. Their fix is LoRA, which caps update strength by confining the update to a rank-r subspace. That is a blunt instrument: it buys protection by giving up full-rank updates. The obvious question is whether the damage can be bounded directly.

The answer we obtained is: **not by the mechanism we proposed.** The anchored trust region the optimizer is named after contributes nothing, at four scales in two formulations. And at the scale we could measure, the problem it was built to solve does not clearly exist — Muon *ties* AdamW when fine-tuning here, so there was no deficit to close.

What the work produced instead is the leverage identity of Section 3.2, the fused-projection defect it exposes, a routing bug affecting every matrix optimizer under weight tying, and a configuration that does beat the state of the art — in the *other* regime, from scratch, on both step and wall-clock budgets. Sections 5 and 6 give the evidence; Sections 3 and 4 give the algorithm and the protocol, both of which stand independently of any result.

2. What the 2025–2026 literature settles

Taken as given, with sources:

Matrix methods beat AdamW from scratch, by about 1.1–1.4×, and the margin shrinks with scale [1]. Any claim of a larger margin should be treated as a claim about tuning.

Matrix whitening decomposes into two ingredients — spectral normalization and variance adaptation — and variance adaptation is the one that explains SOAP's edge over Muon [2]. Muon implements only the first. Every 2026 Muon variant that wins (NorMuon, AdaMuon, Muon², NAMO) adds the second.

Weight decay in a normalized network is not a regularizer; it is an indirect controller of the angular learning rate $\eta_{\text{ang}} = \| \hat{W}_{t+1} - \hat{W}_t \|_F$ [3]. Hyperball makes the control explicit by pinning $\| W \|_F$ and normalizing update length.

Muon implicitly assumes isotropic layer inputs. The steepest-descent direction under a quadratic model is $\text{msgn}(G(ZZ^\top)^{-1})$, with Muon the $ZZ^\top \propto I$ special case [4].

Momentum before orthogonalization is a spectral denoiser — it widens the gap between signal and noise singular values, stabilizing the subspaces the next step depends on. NVIDIA independently found Nesterov did not help at scale and dropped it [7].

The small-singular-value tail is numerical noise, and regularized orthogonalization is an open problem [7, §7].

Matrix methods are geometrically mismatched to some layer types. NVIDIA report accuracy loss and NaNs from orthogonalizing Mamba's `conv1d`, and fall back to AdamW for it [7]; Muon's own reference implementation excludes the stem convolution by hand [8].

3. ASTRO

For a parameter reshaped to the linear operator it represents (Section 3.1), the matrix path is:

1. momentum, $M \leftarrow \beta_1 M + (1-\beta_1)G$;
2. neuron-wise variance adaptation;
3. a spectral filter — approximately setting singular values to one;
4. update-RMS matching;
5. cautious sign masking;
6. norm control, and optionally an anchor at the pretrained weights.

Everything else takes AdamW. Setting `variance="none"`, `cautious=False`, `anchor=False` and `norm_control="none"` recovers Muon exactly; that reduction is asserted in the test suite, which is what makes the ablation in Section 5.3 mean anything.

3.1 Routing, and a bug worth naming

Spectral optimizers assume the parameter *is* a linear operator, so that setting its singular values to one says something about the map it computes. That assumption is false for a surprising number of tensors.

kind	
depthwise conv $(C, 1, k, k)$	reshaping to (C, k^2) makes a matrix whose rows are unrelated filters on disjoint cha
stem conv $(C, \leq 4, k, k)$	
norms, biases, gains	
embeddings, classifier heads	rows are inde

The bug. GPT-2 ties `lm_head.weight` to `transformer.wte.weight` — they are one tensor, so `named_parameters()` reports it *once*, under the `wte` name. Consequently a name-keyed router never sees `lm_head.weight`, and the tied token embedding stays on the spectral path. The position embedding `wpe`, being shape (block, n_{embd}), is indistinguishable from a

dense matrix by shape alone and goes the same way. Worse, the usual fallback heuristic — "the last 2-D parameter is the classifier head" — then selects the final block's **MLP projection**, a genuine operator, and excludes it.

Three tensors misrouted, in both directions. This is not specific to ASTRO: Muon and NorMuon share the router, so the bug changes what *every* matrix optimizer does on a weight-tied transformer. It raises nothing and costs accuracy silently. The fix is to resolve parameters by identity rather than name, and to detect `nn.Embedding` and the output head structurally from the module graph.

3.2 Spectral filters and the noise tail

Proposition 1. *For* $X = U\Sigma V^\top$ *and any odd polynomial* $p(x) = ax + bx^3 + cx^5$,

$$p(X) := aX + bX(X^\top X) + cX(X^\top X)^2 = U\,\mathrm{diag}(p(\sigma))\,V^\top.$$

A composition of k *such steps applies* $p_k \circ \cdots \circ p_1$ *to every singular value, using only matrix multiplications.*

Choosing an optimizer's spectral behaviour therefore reduces to choosing a scalar function on $[0, 1]$ — and gives a free test oracle: the same recurrence on plain scalars must reproduce the singular values of the matrix path, which the tests assert to 10^{-4}.

Muon's quintic is $(a, b, c) = (3.4445, -4.7750, 2.0315)$, chosen to maximize the slope at zero. Since $a > 1$ the origin is *repelling*, by design — but the smallest singular directions are dominated by floating-point noise, and they get amplified with everything else. Measured, on a 48×72 matrix with 6 planted signal directions and 42 noise directions at $\sigma = 0.004$:

	signal	noise
input (normalized)	0.486 … 0.243	0.0019
after Muon, 5 steps	0.973	**0.839**
after dead-zone filter, 10 steps	**1.000**	**0.000000**

Muon lifts pure noise by $430\times$, to 86% of the magnitude it gives real signal.

Proposition 2 (threshold–decay coupling). *Let* $p(\sigma) = a\sigma + b\sigma^3 + c\sigma^5$ *satisfy* $p(1) = 1$, $p(\tau) = \tau$ *for* $\tau \in (0, 1)$, *and* $|p'(1)| \le 1$. *Then* $a = 1 - O(\tau^2)$.

So a stationary iteration that both fixes the head at 1 and has a dead zone at τ has slope at the origin approaching 1 from below — and suppressing the tail by $100\times$ at $\tau = 0.1$ would need ≈ 454 iterations. The obstruction is an artifact of *stationarity*: a **non-stationary** 10-step composition, found by minimax search, realizes a step at $\sigma \approx 0.11$ with pass-band error 8×10^{-6} and stop-band leakage 1.5×10^{-6}, with all intermediate iterates bounded by 1.0000002.

3.3 Update-RMS matching

Under isotropic stationary gradients the expected squared update norms are $\frac{1-\beta_1}{1+\beta_1}mn$ for Adam and $\min(m, n)$ for a polar factor, so matching them requires scaling the polar factor by

$$\sqrt{\frac{1-\beta_1}{1+\beta_1}} \cdot \sqrt{\max(m, n)}.$$

At $\beta_1 = 0.9$ the first factor is 0.229, recovering the constant 0.2 used by Moonlight and by NVIDIA [7]. This is a *fairness* requirement, not a performance trick: without it a learning rate cannot transfer between the spectral and scalar paths, and every comparison against AdamW silently confounds "better algorithm" with "different effective step size". The ablation (Section 5.3) finds it is also the single most valuable component.

3.4 Cautious masking

Orthogonalization is a *global* operation on a matrix: it equalizes singular values without regard to any individual coordinate. It therefore produces entries that point uphill on the current batch — we measure 5–15% of coordinates disagreeing in sign with the gradient. Masking them [9] restores a guaranteed descent direction,

$$\langle\, u \odot \mathbb{1}[u \odot g > 0],\ g \,\rangle = \sum_{i:\, u_i g_i > 0} u_i g_i \ \geq\ 0,$$

whereas $\langle u, g \rangle$ need not be positive when $u = \mathrm{msgn}(M)$ and the momentum has drifted from the current gradient. Surviving coordinates are rescaled by numel/count so the mask changes direction without acting as a hidden learning-rate cut.

3.5 The anchor: two formulations

Hard. Project onto a ball around the pretrained weights, $W \leftarrow W_0 + D \cdot \min(1, \rho/d)$ with $D = W - W_0$ and $d = \|D\|_F / \|W_0\|_F$. Projection onto a Frobenius ball is a radial rescale, so this costs one norm and one blend.

Elastic. A restoring force, $W \leftarrow W - \eta\kappa(W - W_0)$: decoupled weight decay toward W_0 rather than toward the origin, i.e. L2-SP [10] applied to a spectral optimizer.

The distinction turned out to matter, and not in the way intended. A hard projection is a *budget*: once the iterate reaches the boundary the constraint stops all further outward motion, so tightening ρ does not reduce update strength — it converts training into early stopping at a fixed distance. A restoring force has no budget to exhaust. Elastic duly beat hard by 2.2%. Neither beat having no anchor at all (Section 6.2).

Scope. The constraint must apply to the *concatenated* parameter vector, not per tensor. A per-tensor cap forces every tensor to the same relative drift, destroying the non-uniform allocation a good optimizer discovers — measured, AdamW spends drift unevenly (0.08 on a depthwise kernel, 0.23 on the stem and dense conv), and flattening that profile costs accuracy even under a generous total budget. Tightening a per-tensor ρ made the *unconstrained* stem and head drift **more** (0.47 → 0.59): the constraint relocates update strength rather than reducing it.

3.6 The algorithm

```
Algorithm 1  ASTRO step
─────────────────────────────────────────────────────────────────────────
Input: lr η, betas (β₁,β₂), eps ε, weight decay λ, filter Φ,
       flags: variance axis, placement, cautious, norm control, anchor

for each parameter p:
    if route(p) is not an operator:            ▷ §3.1
        AdamW(p);  continue

    W ← matrix_view(p);  G ← matrix_view(∇p)
    M ← β₁M + (1-β₁)G                          ▷ momentum first: denoise

    if placement ∈ {pre, both}:                ▷ §3.2 conditioning
        v ← β₂v + (1-β₂)·rowmean(G²)
        M̃ ← M / (√(v/(1-β₂ᵗ)) + ε)
    else: M̃ ← M

    D ← Φ(M̃)                                   ▷ Prop. 1; Muon or dead-zone

    if placement ∈ {post, both}:               ▷ NorMuon placement
        w ← β₂w + (1-β₂)·rowmean(D²)
        D ← (D / (√(w/(1-β₂ᵗ)) + ε))·(‖D‖/‖·‖) ▷ norm-preserving

    if cautious:  D ← D ⊙ 1[D⊙G>0] · numel/count      ▷ §3.4
    D ← D · √((1-β₁)/(1+β₁))·√(max(m,n))              ▷ §3.3

    if norm control = wd:   W ← (1-ηλ)W
    W ← W - ηD
    if norm control = hyperball:  W ← R·W/‖W‖

    if anchor = elastic:  W ← W - ηκ(W - W₀)          ▷ §3.5
project globally if anchor = hard                     ▷ §3.5 scope
```

4. How the comparison is made hard to rig

The protocol is enforced in code, in `astro/bench/protocol.py`, not by convention.

- **Equal tuning budget.** 16 trials per optimizer, log-uniform, from an RNG seeded identically per optimizer so draw k is equally lucky for everyone. `tune()` **raises** if the optimizers do not all tune the same *number* of hyperparameters — three each here. Sweeping the proposed method over three knobs and the baseline over one is the commonest way a result is inflated, so it is a hard error rather than a guideline.
- **Tuning and evaluation are separated.** Tuning runs on seed 0; the winner is re-run on seeds 100–104, so tuning cannot select on the noise it is scored against.
- **Paired statistics.** Seed-by-seed differences with percentile bootstrap intervals, not comparisons of independent means. A difference that does not survive the interval is not reported as a difference.
- **Wall-clock beside step counts.** A per-step win that costs more time than it saves is recorded as a loss.

- **Tuning traces published.** A best-so-far curve still descending at trial 16 is direct evidence of under-tuning, and is printed rather than hidden.
- **No dead knobs.** Where a variant disables the component its third hyperparameter controls, the budget is spent on a live one instead, so no variant is handicapped by tuning a parameter that does nothing.

4.1 Model, data, and what the scale permits

The model is nanoGPT's GPT-2 [8], vendored with the training and sampling machinery removed and nothing else changed: pre-norm blocks, fused QKV, 4×-expansion GELU MLP, learned position embeddings, weight tying, and the $N(0, 0.02/\sqrt{2L})$ scaled residual initialization. Only the shape is reduced.

	benchmark	GPT-2 (124M)
layers / heads / width / context	4 / 4 / 128 / 128	12 / 12 / 768 / 1024
non-embedding parameters	806K	124M

Corpora are WikiText-2 (pretraining) and tinyshakespeare (fine-tuning) — encyclopedic prose to Early Modern verse, a large stylistic shift over a shared alphabet. Tokenization is character-level with a vocabulary *shared across both corpora*, because otherwise the embedding table cannot transfer and the fine-tuning task would measure re-initialization rather than adaptation. Character rather than BPE because a 50257-row embedding at width 128 would hold an order of magnitude more parameters than the transformer blocks, all of them routed to the scalar path — the benchmark would mostly measure AdamW on a lookup table.

Calibration. A fine-tuning task can only detect feature disruption if the pretrained initialization matters. Measured under matched AdamW: **2.146** validation loss from the pretrained checkpoint against **2.622** from random initialization, a 0.48-nat gap. Over the full 1M-token corpus that gap is only 0.23, because there is enough data to relearn from scratch; the 40K-token pool is what makes the task sensitive.

What 806K parameters permits. Not a claim about 124M, and certainly not about frontier scale. Wen et al. [1] specifically find optimizer advantages shrink with scale, so a small-scale win is weak evidence and a small-scale *null* is weak evidence too. Section 7 states the boundary; `scripts/bench_gpu_llm.py` runs the identical protocol at real GPT-2 sizes with real BPE for anyone with a GPU.

5. Results

5.1 GPT-2 fine-tuning — the target regime

AdamW-pretrained on WikiText-2, then fully fine-tuned on tinyshakespeare. 16 tuning trials per optimizer at three hyperparameters each, best configuration re-run on 5 seeds. Lower is better; a negative Δ favours the row.

The two comparisons that matter:

optimizer	val loss (mean ± sd)	vs control	paired Δ [95% CI]	sig	s/run
`astro`	2.1779 ± 0.0394	-0.51%	[-0.0176, -0.0047]	**yes**	16.2
`adamw` *(control)*	2.1890 ± 0.0436	—	—	—	11.7
`muon`	2.1893 ± 0.0442	+0.01%	[-0.0057, +0.0062]	no	15.3

Full field:

optimizer	val loss (mean ± sd)	vs control	paired Δ [95% CI]	sig	s/run
`astro`	2.1779 ± 0.0394	-0.51%	[-0.0176, -0.0047]	**yes**	16.2
`cautious`	2.1812 ± 0.0384	-0.36%	[-0.0130, -0.0037]	**yes**	12.4
`adamw` *(control)*	2.1890 ± 0.0436	—	—	—	11.7
`muon`	2.1893 ± 0.0442	+0.01%	[-0.0057, +0.0062]	no	15.3
`normuon`	2.1894 ± 0.0458	+0.02%	[-0.0072, +0.0068]	no	19.3
`soap`	2.1941 ± 0.0387	+0.23%	[-0.0022, +0.0126]	no	20.1
`ademamix`	2.2037 ± 0.0341	+0.67%	[+0.0057, +0.0236]	**yes**	12.5
`sgd`	2.2457 ± 0.0499	+2.59%	[+0.0416, +0.0790]	**yes**	11.3

Selected configurations:

optimizer	selected configuration
`adamw`	`lr` =0.0006935, `weight_decay` =0.2312, `beta2` =0.9968
`muon`	`lr` =0.005492, `adamw_lr` =0.0008177, `weight_decay` =5.952e-05
`astro`	`lr` =0.001771, `scalar_lr_mult` =0.1978, `anchor_strength` =0.1655

Tuning traces (best-so-far):

- `adamw` : 2.150 → 2.150 → 2.150 → 2.149 → 2.149 → 2.149
- `sgd` : 2.487 → 2.323 → 2.323 → 2.323 → 2.208 → 2.208
- `muon` : 2.170 → 2.156 → 2.156 → 2.156 → 2.156 → 2.156
- `normuon` : 2.166 → 2.156 → 2.156 → 2.156 → 2.156 → 2.156
- `soap` : 2.154 → 2.154 → 2.154 → 2.154 → 2.154 → 2.154
- `ademamix` : 2.167 → 2.155 → 2.155 → 2.155 → 2.155 → 2.155
- `cautious` : 2.145 → 2.145 → 2.145 → 2.138 → 2.138 → 2.138
- `astro` : 2.197 → 2.156 → 2.152 → 2.152 → 2.152 → 2.152

5.2 GPT-2 from scratch — the held-out regime

From random initialization on WikiText-2. This is the regime the Muon/SOAP literature is built on, where matrix methods are expected to win, and it is held out from the design decisions in Section 6.

optimizer	val loss (mean ± sd)	vs control	paired Δ [95% CI]	sig	s/run
`astro`	1.8309 ± 0.0070	-17.74%	[-0.4180, -0.3728]	**yes**	48.8
`muon`	1.8310 ± 0.0152	-17.73%	[-0.4279, -0.3628]	**yes**	44.2
`adamw` *(control)*	2.2257 ± 0.0313	—	—	—	36.4

Full field:

optimizer	val loss (mean ± sd)	vs control	paired Δ [95% CI]	sig	s/run
`normuon`	1.8072 ± 0.0144	-18.80%	[-0.4529, -0.3854]	**yes**	48.0
`astro`	1.8309 ± 0.0070	-17.74%	[-0.4180, -0.3728]	**yes**	48.8
`muon`	1.8310 ± 0.0152	-17.73%	[-0.4279, -0.3628]	**yes**	44.2
`soap`	1.8783 ± 0.0107	-15.61%	[-0.3717, -0.3232]	**yes**	50.1
`cautious`	2.1264 ± 0.0318	-4.46%	[-0.1174, -0.0841]	**yes**	37.6
`adamw` *(control)*	2.2257 ± 0.0313	—	—	—	36.4
`ademamix`	2.2460 ± 0.0222	+0.91%	[+0.0030, +0.0413]	**yes**	34.4
`sgd`	2.3853 ± 0.0044	+7.17%	[+0.1346, +0.1845]	**yes**	59.6

Tuning traces (best-so-far):

- `adamw` : 2.247 → 2.247 → 2.219 → 2.219 → 2.219 → 2.211 ← still descending at the budget limit
- `sgd` : 2.634 → 2.439 → 2.439 → 2.439 → 2.387 → 2.387
- `muon` : 2.064 → 1.850 → 1.850 → 1.850 → 1.847 → 1.830 ← still descending at the budget limit
- `normuon` : 2.036 → 1.839 → 1.839 → 1.839 → 1.838 → 1.817 ← still descending at the budget limit
- `soap` : 2.002 → 1.897 → 1.897 → 1.897 → 1.892 → 1.892
- `ademamix` : 2.279 → 2.215 → 2.215 → 2.215 → 2.215 → 2.215
- `cautious` : 2.205 → 2.184 → 2.111 → 2.111 → 2.111 → 2.106 ← still descending at the budget limit
- `astro` : 2.213 → 1.930 → 1.925 → 1.853 → 1.842 → 1.836 ← still descending at the budget limit

5.3 Component ablation on GPT-2 fine-tuning

Each variant reverts exactly one component, and each still tunes three hyperparameters, with the third knob following the anchor configuration so no variant wastes budget on an inert parameter. A **positive** Δ means the full method is better than the variant, i.e. the removed component was contributing.

optimizer	val loss (mean ± sd)	vs control	paired Δ [95% CI]	sig	s/run
`astro_no_anchor`	2.1777 ± 0.0390	-0.01%	[-0.0006, +0.0001]	no	14.5
`astro_full` *(control)*	2.1779 ± 0.0394	—	—	—	17.2
`astro_variance_pre`	2.1779 ± 0.0394	+0.00%	[+0.0000, +0.0000]	no	16.2
`astro_no_variance`	2.1780 ± 0.0373	+0.00%	[-0.0021, +0.0034]	no	14.5
`astro_variance_both`	2.1798 ± 0.0279	+0.09%	[-0.0101, +0.0139]	no	16.4
`astro_hyperball`	2.1848 ± 0.0365	+0.32%	[+0.0034, +0.0094]	**yes**	15.6
`astro_deadzone`	2.1857 ± 0.0442	+0.36%	[-0.0015, +0.0148]	no	18.8
`astro_no_cautious`	2.1938 ± 0.0354	+0.73%	[+0.0111, +0.0207]	**yes**	15.9
`astro_anchor_hard`	2.1938 ± 0.0245	+0.73%	[+0.0002, +0.0309]	**yes**	15.2
`astro_no_rms`	2.1985 ± 0.0484	+0.95%	[+0.0112, +0.0294]	**yes**	15.8

5.4 Candidate rounds (post-hoc, from scratch)

Variants proposed after the field was measured, each tuned under the same budget and RNG stream the field received. A **negative** Delta favours the candidate. Exact paired Wilcoxon, Holm-corrected, 10 seeds.

comparison	paired Delta	Cohen's d	Holm-adj. p
astro_normuon_cautious vs adamw	-0.4297	-14.36	**0.0117**
astro_split_normuon vs adamw	-0.4297	-14.36	**0.0176**
astro_normuon_replica vs adamw	-0.4138	-13.30	**0.0117**
astro_split vs adamw	-0.4128	-11.01	**0.0176**
astro_nesterov_muonscale vs adamw	-0.3956	-12.35	**0.0176**
astro_nosplit vs adamw	-0.3887	-10.83	**0.0176**
astro_post vs adamw	-0.3738	-8.68	**0.0117**
astro_post_nocautious vs adamw	-0.3605	-10.61	**0.0117**
astro_normuon_cautious vs muon	-0.0407	-2.81	**0.0117**
astro_split_normuon vs muon	-0.0407	-2.81	**0.0176**
astro_normuon_replica vs muon	-0.0248	-1.64	**0.0117**
astro_split vs muon	-0.0238	-1.28	**0.0293**
astro_normuon_cautious vs normuon	-0.0138	-1.29	**0.0195**
astro_split_normuon vs normuon	-0.0138	-1.29	**0.0293**
astro_nesterov_muonscale vs muon	-0.0066	-0.51	0.3203
astro_nosplit vs muon	+0.0003	+0.01	1.0000
astro_normuon_replica vs normuon	+0.0021	+0.19	0.7695
astro_split vs normuon	+0.0031	+0.17	1.0000
astro_post vs muon	+0.0152	+0.62	0.1875
astro_nesterov_muonscale vs normuon	+0.0203	+1.69	**0.0176**
astro_nosplit vs normuon	+0.0272	+1.37	**0.0293**
astro_post_nocautious vs muon	+0.0285	+1.56	**0.0234**
astro_post vs normuon	+0.0431	+2.19	0.1875
astro_post_nocautious vs normuon	+0.0646	+3.90	0.1875

5.5 Equal wall-clock, from scratch

Every optimizer runs for the same number of seconds rather than the same number of steps, so a cheaper step converts directly into more of them. Measured on an idle machine; the reference is `normuon` at its natural step count.

optimizer	ms/step	steps in budget	val loss
`astro_normuon_cautious`	91.0	405	1.7946
`normuon`	92.2	400	1.8109
`muon`	84.1	438	1.8117
`astro_split`	90.2	409	1.8143
`adamw`	62.3	592	2.0758

6. Five things measurement changed

6.1 The premise did not reproduce

ASTRO exists because Qu et al. [5] find Muon loses to Adam when fine-tuning Adam-pretrained weights. At this scale, it does not: Muon *ties* AdamW, with a paired interval straddling zero. There was no deficit for the fix to close, which is the most likely reason none of the anchor formulations could show a benefit. At 806K parameters with a character vocabulary there is far less pretrained structure to disrupt than in the models Qu et al. studied. This does not refute their result — it bounds where the ASTRO design premise applies, and the bound excludes the regime we could afford to measure.

6.2 The anchor failed three times

scale	formulation	outcome
CNN, synthetic shift	hard, per-tensor $\rho = 0.10$	1.746 vs 1.195 unanchored
CNN, synthetic shift	hard, global $\rho = 0.12$	1.335 vs 1.195 unanchored
GPT-2 fine-tuning	hard, ρ tuned	tuner selected $\rho = 0.77$ of 1.0 — nearly non-binding
CNN ablation	elastic, κ tuned	-0.0001 vs no anchor; inert

Every configuration that made the constraint bind was worse than leaving it off, and whenever the tuner was free to choose, it chose to switch the component off. The component the optimizer is named after does not work. It ships disabled by default, and the name is now a historical artifact rather than a description.

6.3 A theoretically appealing change that was wrong

We moved variance adaptation from before orthogonalization to after, on the argument that a filter setting every singular value to one must erase a pre-filter rescale — so the rescale should be applied where it survives, as NorMuon does. The ablation disagreed: `pre` and `both` tie each other and both beat `post` by 0.8%. Conditioning the matrix handed to the polar iteration is doing the work, exactly as Muon² claims, and NorMuon's placement does not stack on top of it here. The default was changed back.

Implementing the change surfaced two bugs that no passing test would have caught, and they are worth recording because both are easy to write:

1. taking the second moment of the **gradient** and applying it to the **orthogonalized update** — quantities differing by orders of magnitude, which needs two separate buffers;
2. dividing by a per-row scale **without renormalizing**, which is unbounded: a neuron whose gradient goes quiet produces a denominator near ε and an update thousands of times too large. Pre-placement is shielded from this by the filter that follows it; post-placement has nothing after it and must normalize itself.

Fixing them moved ASTRO from +20.1% to +2.5% against AdamW on the CNN fine-tuning task. The first version was caught because a wiring check regressed catastrophically and the arithmetic was checked against NorMuon's reference rather than attributed to tuning noise.

6.4 Cautious masking works, and it is nearly free

The one component added late that earned its place. Removing it costs 3.2% in the ablation. Its justification is structural rather than empirical — orthogonalization is a global operation and demonstrably produces uphill coordinates — and it costs one elementwise comparison.

6.5 The component ranking is not the one the design assumed

Measured, on the CNN fine-tuning ablation:

component removed	cost
update-RMS matching	**+5.3%**
Hyperball instead of weight decay	+4.5%
cautious masking	+3.2%
hard anchor instead of elastic	+2.2%
dead-zone filter enabled	+1.9%
variance adaptation	+1.1%
the anchor	+0.0%

The most valuable component is the one that exists for *fairness* — making a learning rate transfer between the spectral and scalar paths. That is worth stating plainly: a large part of what a matrix optimizer appears to contribute is step-size calibration, and an evaluation that does not control for it will credit the calibration to the matrix structure.

7. Limitations

Scale. 806K non-embedding parameters, on four CPU cores. Nothing here tests the regime where [1,3,7] make their claims, and optimizer rankings are known to change with scale. Treat the from-scratch results as evidence the implementations are correct, and the fine-tuning results as evidence about small models on small data.

Character-level tokenization keeps the spectral path dominant, which is what we wanted to measure, but it is not how GPT-2 is trained.

Three tuned hyperparameters is a modest budget. Equal across optimizers, which is what fairness requires, but 16 trials over 3 dimensions leaves every method somewhat under-tuned. The traces are printed so this is visible; where a trace is still descending at trial 16 the corresponding number is an upper bound on that optimizer's deficit, not an estimate of it.

`gpt_finetune` is in-sample. Observing that Cautious-AdamW won on it is what prompted adding cautious masking to ASTRO. Its result is therefore not independent evidence for that component; `gpt_scratch` and the CNN tasks are the held-out checks.

SOAP is the published algorithm, not NVIDIA's KL-SOAP. The KL variant's changes fix a large-batch instability that does not arise at CPU batch sizes; implementing a variant imprecisely is worse than implementing the canonical algorithm correctly.

The dead-zone filter's noise model is a planted-spectrum experiment, not a measurement of where real gradient spectra become noise-dominated, and its threshold τ was not tuned per task.

Wall-clock caveat. Some benchmark stages ran while the test suite was executing on the same four cores. Loss numbers are unaffected — every task is seeded and deterministic — but individual `s/run` figures may be inflated by contention, so wall-clock should be read as approximate.

8. Falsification outcomes

Criteria fixed before the experiments ran.

- **Trust region.** *Criterion:* beat tuned AdamW on fine-tuning across seeds with non-overlapping intervals. **Outcome: failed, at three scales in two formulations.** Reported, and the component ships off. What the investigation produced instead is Section 3.5 — the scope result and the budget-versus-restoring-force distinction — which is a real finding.
- **Dead-zone filter.** *Criterion:* beat `dead_zone=0` at equal budget on end-task loss. **Outcome: failed** (+1.9%). Its *spectral* behaviour (Section 3.2) is established independently and remains the stronger result: it solves a problem named as open in [7], at 10 iterations rather than the 454 Proposition 2 shows a stationary iteration would need. It ships off.
- **ASTRO overall.** *Criterion:* beat tuned AdamW and Muon on GPT-2 fine-tuning. **Outcome:** Section 5.
- **Wall-clock.** A per-step win costing more time than it saves is a loss. Recorded as such.

9. Related work

Muon [8] and the modular-duality framework that explains it [6]; Shampoo/SOAP-family whitening and the decomposition of its benefit into spectral normalization plus variance adaptation [2]; Hyperball's explicit control of the angular learning rate [3]; Newton-Muon's correction for anisotropic layer inputs [4]; NVIDIA's scaling study, which supplies both the update-RMS matching framework and the open problem Section 3.2 addresses [7]; the optimizer-mismatch result that motivates the exercise [5]; cautious optimizers [9]; L2-SP [10]; and the benchmarking discipline this paper tries to honour [1].

References

1. K. Wen, D. Hall, T. Ma, P. Liang. *Fantastic Pretraining Optimizers and Where to Find Them.* arXiv:2509.02046.
2. K. Frans et al. *What Really Matters in Matrix-Whitening Optimizers?* arXiv:2510.25000.
3. K. Wen, X. Dang, K. Lyu, T. Ma, P. Liang. *Fantastic Pretraining Optimizers II: Hyperball Optimization.* arXiv:2606.16899.
4. Z. Du, W. Su. *The Newton–Muon Optimizer.* arXiv:2604.01472.
5. X. Qu et al. *Can Muon Fine-tune Adam-Pretrained Models?* ICML 2026, arXiv:2605.10468.
6. J. Bernstein, L. Newhouse. *Modular Duality in Deep Learning.* arXiv:2410.21265.
7. M. Khona et al. (NVIDIA). *SOAP, Muon, and Beyond: Pushing LLM Pretraining Scales.* arXiv:2607.20548.
8. K. Jordan et al. *Muon: An optimizer for hidden layers in neural networks*; A. Karpathy, *nanoGPT*.
9. K. Liang et al. *Cautious Optimizers: Improving Training with One Line of Code.* arXiv:2411.16085.
10. X. Li, Y. Grandvalet, F. Davoine. *Explicit Inductive Bias for Transfer Learning with Convolutional Networks.* ICML 2018.
11. N. Vyas et al. *SOAP: Improving and Stabilizing Shampoo using Adam.* arXiv:2409.11321.